Especificación del servicio · caja negra (Reto 2)

Permite reimplementar el servicio sin tocar el original. Cada comportamiento lleva su evidencia: los `id` de `experimentos.jsonl`.

Confianza: `confirmado` = se probó y se repite · `probable` = se vio una vez o se infiere · `supuesto` = no se probó y es lo que hace nuestro reemplazo por defecto.

Los identificadores `n` de los 20 ejemplos son del material (`material/pruebas/ejemplos.jsonl`). Los `id` con dos rangos citan la grabación original (v1) y su rejuego contra la versión vigente (v2).

1. Entrada

Rutas, métodos, autenticación, forma del cuerpo y límites.

ID	Comportamiento	Evidencia	Confianza
ENT-1	Existen exactamente tres rutas: `POST /msg`, `GET /health` y `POST /__reset`. Cualquier otro método o ruta responde `404 {"error":"E404","mensaje":"ruta desconocida"}`	2309, 2311, 2312, 2313, 2320, 2321, 2325, 2329, 2332	confirmado
ENT-2	La ruta se resuelve **antes** del token: una ruta desconocida responde 404 incluso sin `Authorization`	2335, 2338, 2339	confirmado
ENT-3	La barra final no es la ruta: `POST /msg/` es 404	2325	confirmado
ENT-4	El query string no es la ruta: `GET /msg?x=1` es 404	2321	confirmado
ENT-5	Autenticación: `Authorization: Bearer <token>`, con espacio y el token exacto. Sin header, con token ajeno o sin el prefijo `Bearer` → `401 {"error":"E001","mensaje":"no autorizado"}`	2323, 2326, 2327, 2328	confirmado
ENT-6	La autenticación ocurre **antes** de leer el cuerpo: JSON roto o `{}` sin token dan 401 y no 400	2336, 2337	confirmado
ENT-7	`GET /health` con token válido → `200 {"ok":true}`; sin token → 401 E001	2331, 2330	confirmado
ENT-8	`POST /__reset` con token válido → `200 {"ok":true}`; sin token → 401 E001	2333, 2334	confirmado
ENT-9	El cuerpo de `POST /msg` es `{"m": "..."}`. Sin cuerpo o con JSON malformado → `400 {"error":"E100","mensaje":"json inválido"}`	2315, 2316	confirmado
ENT-10	Si falta `m` o no es texto (`{"m": 5}`) → `400 {"error":"E102","mensaje":"falta m"}`	2317, 2318	confirmado
ENT-11	El `Content-Type` se ignora: un cuerpo JSON enviado como `text/plain` se acepta	2324	confirmado
ENT-12	Los headers de respuesta no se comparan; todas las respuestas conocidas son JSON (`application/json; charset=utf-8`)	2309, 2324, 2331, 2333	confirmado

2. Integridad

Cómo valida el servicio que un mensaje está bien formado. La evidencia de cada regla es un mensaje que la aísla.

ID	Comportamiento	Evidencia	Confianza
INT-1	Sin `;` no hay control: `{"m":"PING"}` y `{"m":""}` → `400 {"error":"E103","mensaje":"falta control"}`	2340, 2341	confirmado

ID	Comportamiento	Evidencia	Confianza
INT-2	Algún segmento vacío → `400 {"error":"E104","mensaje":"segmento vacío"}`, incluso si el campo vacío es el último	2346	confirmado
INT-3	Control presente pero no numérico → `400 {"error":"E201","mensaje":"control mal formado"}`	2367	confirmado
INT-4	Control numérico que no es el del mensaje → `400 {"error":"E202","mensaje":"control inválido"}`	2343, 2344, 2352	confirmado
INT-5	Comando inexistente → `400 {"error":"E301","mensaje":"comando desconocido "}` (el espacio final es del original y se conserva)	2362, 2350	confirmado
INT-6	Cantidad de argumentos distinta de la declarada → `400 {"error":"E302","mensaje":"aridad inválida"}`	2345, 2342, 2347	confirmado
INT-7	El comando no distingue mayúsculas (`ping;64` y `CONSULTa;AC-0001` funcionan)	2349, 2361	confirmado
INT-8	El espacio no se recorta: `" PING;64"` es un comando desconocido	2350	confirmado
INT-9	Orden medido: `E103` y `E104` preceden al control; `E202` precede a `E301` y a `E302`	2340, 2346, 2343, 2344	confirmado
INT-10	Orden entre `E201` y `E103`/`E104` no se probó en combinación (un control mal formado junto a un segmento vacío). Nuestro reemplazo evalúa `E103` → `E104` → `E201` → `E202` → `E301` → `E302` → negocio	—	supuesto
INT-11	Un monto no entero (por ejemplo `DEPOSITO;AC-0001;abc;…`) no se probó. Nuestro reemplazo responde `E302`	—	supuesto

3. Comandos

Un bloque por comando: argumentos, efecto y respuesta. La aridad no cuenta el comando ni el control. Los `id` citan el ejemplo del material y su rejuego contra la versión vigente.

PING

- **Argumentos**: ninguno. **Efecto**: ninguno. **Respuesta**: `200 {"pong": true}`.
- Evidencia: n=1 (id 2351 v1, 2395 v2); 2349; 2350. **Confianza**: confirmado.

ALTA

- **Argumentos**: un nombre. **Efecto**: crea la cuenta `AC-NNNN` con saldo 0; el contador de cuentas avanza sólo si la operación prospera. **Respuesta**: `201 {"cuentaId":"AC-NNNN"}`.
- Evidencia: n=3 (2353 / 2397), n=4 (2354 / 2398). **Confianza**: confirmado.
- No se probó si el nombre es único ni qué pasa al repetirlo: **supuesto** (nuestro reemplazo permite repetirlo y crea otra cuenta).

CONSULTA

- **Argumentos**: un id de cuenta. **Respuesta**: `200 {"saldo":"NNNN.NN","cuenta_id":"AC-NNNN"}` (el saldo es texto decimal en unidades, dos decimales). Si la cuenta no existe: `404 {"error":"E401","mensaje":"cuenta inexistente"}`.
- Evidencia: n=5 (2355 / 2399), n=8 (2358 / 2402), n=15 (2365 / 2409). **Confianza**: confirmado.

DEPOSITO

- **Argumentos**: id de cuenta y monto en centavos. **Efecto**: acredita el monto; el monto viaja en centavos (`12345` se informa como `"123.45"`). **Respuesta**: `200 {"status":"recieved","operacionId":"OP-NNNN"}` (con la errata `recieved`, que es del original).
- Evidencia: n=6 (2356 / 2400), n=18 (2368 / 2412), 3078. **Confianza**: confirmado.

- El tope de 50000 **no** se aplica a los depósitos: un depósito de 250000 fue aceptado (2356).

TRANSFER

- **Argumentos**: cuenta origen, cuenta destino y monto en centavos. **Efecto**: debita y acredita en la misma operación. **Respuesta**: `200 {"status":"recieved","operacionId":"OP-NNNN"}`, o `422 {"error":"E506","mensaje":"límite excedido"}` si el monto supera el tope, o `422 {"error":"E507","mensaje":"saldo insuficiente"}` si el origen no alcanza.
- Evidencia: n=7 (2357 / 2401), n=9 (2359 / 2403), n=16 (2366 / 2410), n=10 (2360 / 2404), 3147, 3204, 2444, 2445, 2446. **Confianza**: confirmado.
- Tope medido: **50000** por transferencia (50000 aceptado, 50001 rechazado), **no acumulado** (tres transferencias seguidas de 40000 se aceptaron) y **no aplicado a los depósitos**.

ANULA

- **Argumentos**: un id de operación. **Efecto**: revierte el movimiento original (devuelve el saldo).

Respuesta: `200 {"ok":true,"anulada":"OP-NNNN"}`; si ya estaba anulada, `200 {"ok":false,"motivo":"ya anulada"}`; si la operación no existe, `404 {"error":"E402","mensaje":"operación inexistente"}`.

- Evidencia: n=13 (2363 / 2407), n=14 (2364 / 2408), 2407, 2408. **Confianza**: confirmado.

4. Memoria entre requests

ID	Comportamiento	Evidencia	Confianza
MEM-1	El estado es por token: cuentas, saldos, operaciones y contadores sobreviven entre requests y son de ese token	2356, 2357, 2358, 2397, 2399	confirmado
MEM-2	Los contadores `AC-` y `OP-` avanzan **sólo** cuando la operación prospera: un error no consume id. La secuencia del material lo muestra (v1 asignó `OP-0004` donde v2, con la transferencia rechazada, asignó `OP-0003`)	2368, 2412	confirmado
MEM-3	`POST /__reset` devuelve el token al estado inicial: sin cuentas y con los contadores en 1	2333, 2395, 2396, 2397	confirmado
MEM-4	Una transferencia mueve las dos cuentas; ANULA revierte el efecto	2358, 2363, 2402, 2407	confirmado
MEM-5	Aislamiento entre tokens: lo de un token no es visible ni modificable desde otro	—	supuesto
MEM-6	Qué pasa si se deposita o transfiere contra una cuenta inexistente: un depósito da `E401` (n=15 lo muestra para CONSULTA; el depósito no se probó)	2365	probable

5. Errores

Cada error con su status, su cuerpo exacto y el orden en que se evalúa.

Código	Status	Cuerpo exacto	Evidencia	Confianza
E001	401	`{"error":"E001","mensaje":"no autorizado"}`	2323, 2326, 2327, 2330, 2334	confirmado
E100	400	`{"error":"E100","mensaje":"json inválido"}`	2315, 2316	confirmado
E102	400	`{"error":"E102","mensaje":"falta m"}`	2317, 2318	confirmado
E103	400	`{"error":"E103","mensaje":"falta control"}`	2340, 2341	confirmado
E104	400	`{"error":"E104","mensaje":"segmento vacío"}`	2346	confirmado
E201	400	`{"error":"E201","mensaje":"control mal formado"}`	2367	confirmado
E202	400	`{"error":"E202","mensaje":"control inválido"}`	2352, 2343, 2344	confirmado
E301	400	`{"error":"E301","mensaje":"comando desconocido "}`	2362, 2350	confirmado

Código	Status	Cuerpo exacto	Evidencia	Confianza
E302	400	`{"error":"E302","mensaje":"aridad inválida"}`	2345, 2342	confirmado
E401	404	`{"error":"E401","mensaje":"cuenta inexistente"}`	2365, 2409	confirmado
E402	404	`{"error":"E402","mensaje":"operación inexistente"}`	2407, 2408	confirmado
E404	404	`{"error":"E404","mensaje":"ruta desconocida"}`	2309, 2311, 2325, 2332	confirmado
E506	422	`{"error":"E506","mensaje":"límite excedido"}`	2359, 2403, 3204	confirmado
E507	422	`{"error":"E507","mensaje":"saldo insuficiente"}`	2366, 2410	confirmado

Orden de evaluación medido, de afuera hacia adentro del mensaje:

1. ruta (E404) antes que token (E001) — 2335 contra 2323;
2. token (E001) antes que el cuerpo (E100/E102) — 2336, 2337;
3. formato del cuerpo (E100, E102) antes que el contenido del mensaje — 2315, 2317;
4. dentro del mensaje: `E103`, `E104`, control (`E201`/`E202`), comando (`E301`), aridad (`E302`),

efecto — 2340, 2346, 2343, 2344, 2345.

6. Cambios durante el día

La versión 2 se desplegó sin aviso a las 15:30. Estos tres comportamientos cambiaron; el resto quedó igual. Lo detecté rejugando lo grabado contra el servicio vivo y sondeando lo que el rejuego dejó ambiguo.

ID	Cambio	v1	v2	Evidencia	Cómo lo detecté
CAM-1	El saldo pasó de entero en centavos a texto decimal en unidades con dos decimales	`{"saldo": 250000}`	`{"saldo": "2500.00"}`	2355 vs 2399, 2443, 3079, 3078 (`12345` → `"123.45"`)	el rejuego de los 20 ejemplos; después un depósito con centavos reales
CAM-2	El tope por transferencia bajó a 50000, sigue siendo por operación y no se aplica a los depósitos	100000 aceptado	50000 aceptado, 50001 → E506	2360 vs 2404, 3147, 3204, 2444, 2445, 2446	búsqueda binaria del tope resolviendo el control de cada monto, y tres transferencias iguales seguidas
CAM-3	Anular una operación inexistente pasó a tener respuesta propia	supuesto `"ya anulada"`	`404 {"error":"E402","mensaje":"operación inexistente"}`	2407, 2408	apareció en el rejuego: la transferencia rechazada dejó a `OP-0003` sin existir

Lo que **no** cambió, y se rejugó para confirmarlo: el frente HTTP completo (48 sondas) y la función de control (los controles grabados siguen sirviendo y `ALTA;Lucia;83` sigue dando `E202`), además de las otras diez filas de errores y los contadores. Evidencia del rejuego: 2415-2438, 2396.

7. Riesgos

ID	Riesgo	Por qué importa	Cómo lo cerraría
RIE-1	**La fórmula del control es desconocida.** Se midió: es función pura del mensaje, insuficiente a mayúsculas, depende del orden de los caracteres, y no es ninguna de las ~30 familias de checksum probadas. Lo que el reemplazo usa hoy es una **tabla de 76 payloads medidos** contra el oráculo	Un mensaje con un control inválido que no esté en la tabla se acepta, y un `E202` esperado no llega: es el mayor costo de fidelidad que queda	Sondear más payloads con el oráculo (hasta 100 requests por payload) o deducir la fórmula con los pares ya medidos
RIE-2	El protocolo no tiene clave de idempotencia del cliente: el `OP-` lo emite el servicio	Un reintento del cliente puede depositar dos veces y el reemplazo no puede evitarlo sin cambiar el cable	Irreducible sin cambiar el protocolo
RIE-3	El original distingue una falla interna de un error de negocio y nuestro reemplazo declara `503 {"error":"no-disponible"}` cuando su propia base no responde	No se sondeó la falla interna del original: el cuerpo es un supuesto nuestro	Provocar una falla y sondear la respuesta del original
RIE-4	El reemplazo no aplica límite de tasa; el original admite 10 req/s y 8000/día por token y marca `X-Frontier-Infra` al pasarse	Un cliente nuevo no debería heredar la ausencia de límite	Poner límite propio sin tocar el contrato
RIE-5	El estado vive en una sola base y el diseño no admite dos escritores	Dos instancias escribiendo el mismo cliente romperían el candado	Un solo proceso escritor, o el candado por cliente que ya usa el reemplazo